

5COSC022W Smart Campus Coursework Report

Answers

This document contains only the written answers requested in the coursework brief.

Part 1.1: JAX-RS Resource Lifecycle

By default, JAX-RS resource classes are request-scoped, which means the runtime usually creates a fresh resource instance for each incoming request. That reduces accidental shared mutable state inside resource classes, but it does not remove concurrency concerns because multiple request-scoped resource instances can still access the same shared backend data structures at the same time. In this coursework, shared state lives inside the in-memory storage service. To keep that safe, the implementation uses a singleton storage layer backed by concurrent maps and synchronized state-changing methods. That design prevents race conditions such as linking a sensor while another request is deleting a room.

Part 1.2: Why HATEOAS Matters

Hypermedia matters because it turns the API into a self-describing navigation surface. A discovery response that advertises links such as rooms and sensors helps clients learn where to go next without relying entirely on static documentation. This reduces coupling, improves maintainability when versions evolve, and makes onboarding easier because the API itself exposes the valid next steps.

Part 2.1: IDs Only vs Full Room Objects

Returning only room IDs reduces payload size and saves bandwidth, especially when the collection becomes large. However, it forces clients to make extra follow-up requests to fetch names, capacities, and linked sensors. Returning full room objects increases payload size slightly, but it reduces client-side complexity and avoids extra round trips. For this coursework, returning full room objects is a sensible default because the domain model is small and richer responses are more useful during testing and demonstration.

Part 2.2: DELETE Idempotency

DELETE is idempotent if repeating the same request does not keep changing the server state after the first successful effect. In this API, deleting an empty existing room removes it and returns 204 No Content. Repeating the same request later returns 404 Not Found, but the room remains absent and the server state does not change again. That still satisfies idempotency. For occupied rooms, repeated deletes continue returning 409 Conflict until the sensor links are removed, so the state also remains stable across retries.

Part 3.1: What `@Consumes(MediaType.APPLICATION_JSON)` Enforces

The annotation declares that the endpoint only accepts JSON payloads. If a client sends a request with a content type such as `text/plain` or `application/xml`, JAX-RS rejects the request before the resource method runs. The usual result is HTTP 415 Unsupported Media Type. This is valuable because the contract stays explicit, invalid formats are blocked consistently by the framework, and the

application does not attempt to deserialize the wrong payload type.

Part 3.2: Why Query Parameters Suit Filtering Better

A URL like `GET /sensors?type=CO2` is better for filtering because the resource is still the sensors collection and type is simply a filter applied to that collection view. Query parameters are better than path parameters for this use case because filters are optional, they scale naturally to multiple search criteria, and they express slicing or searching rather than resource identity. Path parameters are stronger when the value identifies the resource itself, such as `/rooms/LIB-301`.

Part 4.1: Benefits of the Sub-Resource Locator Pattern

The sub-resource locator pattern keeps the parent `SensorResource` focused on sensor-level concerns while delegating reading-history behavior to a dedicated `SensorReadingResource`. This improves maintainability by reducing class size, grouping related behavior together, and making nested functionality easier to test in isolation. In larger APIs, this delegation helps avoid a single oversized controller class containing every nested route and every rule in one place.

Part 5.2: Why 422 Is More Accurate Than 404 Here

HTTP 404 Not Found usually means the target URI does not map to an existing resource. In this case, the URI `/api/v1/sensors` is valid and the endpoint exists. The problem is inside the request body because the referenced `roomId` does not exist. That is why 422 Unprocessable Entity is more precise: the syntax is valid, the request reached the correct endpoint, the body was parsed successfully, but the payload still fails semantic validation because it references a missing related resource.

Part 5.4: Cybersecurity Risk of Exposing Stack Traces

Exposing raw Java stack traces to external clients leaks implementation details that should remain internal. An attacker could learn package names, class names, framework versions, method flow, internal file paths, and hints about weak validation or error boundaries. That information makes reconnaissance easier and can help attackers target specific libraries or exploit patterns. Returning a generic 500 Internal Server Error is safer because it communicates the failure without revealing internal structure.

Part 5.5: Why JAX-RS Filters Are Better for Logging

Logging is a cross-cutting concern, so it is better handled in JAX-RS filters than manually inside every resource method. Filters centralize request and response logging in one place, apply consistently across all endpoints, reduce repeated boilerplate, and keep resource classes focused on business logic. This leads to cleaner architecture and makes later logging changes much easier to maintain.